

ESP32-S3 R503 Fingerprint Sensor Guide

<https://gemini.google.com/app/a18de1831e1112cb>

User prompt: i have platformio project with esp32-s3 processor. I also have two touch sensors bought on aliexpress. first is "SFM-V1.7 Semiconductor Integrated Touch Capacitive Acquisition And Identification Fingerprint Sensor Module UART" and second is "AB25-R503 Circular Round RGB Ring Indicator LED Control DC3.3V MX1.0-6Pin Capacitive Fingerprint Module Sensor" Which one is better? I would like to register different fingers and have different actions based on which finger is presented.

Response: The **AB25-R503** is significantly better for your ESP32-S3 PlatformIO project. It uses the standard **Grow/Synochip UART protocol**, which has native out-of-the-box support in popular open-source libraries like the Adafruit Fingerprint Sensor Library.

Comparison Overview

Feature	AB25-R503 (Recommended)	SFM-V1.7
ESP32 Library Support	Full support (Adafruit Fingerprint Sensor Library)	Scarcely documented, requires custom byte-parsing
RGB Indicator	Built-in 3.3V RGB Ring with software color/breathing controls	Basic or external status pin
Low Power Wakeup	Dedicated TOUCH output pin to wake ESP32-S3 from deep sleep	Varies / tricky interrupt implementation
Power Supply	Natively 3.3V logic & VCC	3.3V logic (varies by sub-board variant)
Enclosure Mounting	Threaded body with mounting nut	Flat board / adhesive mount

How Multi-Finger Action Mapping Works

Both modules store fingerprint templates inside their internal flash memory indexed by an integer ID (ID 1, ID 2, ID 3, up to 200).

To execute different actions per finger:

1. **Enrollment Phase:** Enroll finger #1 (e.g., Right Index) as ID 1, finger #2 (e.g., Right Thumb) as ID 2, and finger #3 (e.g., Left Index) as ID 3.
2. **Recognition Phase:** When a finger is scanned, the sensor runs internal 1:N matching and sends back the matched fingerID and confidence score over UART.
3. **Logic Handling:** In C++, a `switch(fingerID)` block executes your custom functions based on which index was matched.

PlatformIO Implementation (AB25-R503)

1. platformio.ini setup

Add the Adafruit fingerprint library dependency:

```
[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200
lib_deps =
    adafruit/Adafruit Fingerprint Sensor Library @ ^2.1.3
```

2. R503 Pinout & Wiring to ESP32-S3

- **VCC (Red):** 3.3V
- **GND (Black):** GND
- **TX (Yellow):** Connect to ESP32-S3 RX (e.g., GPIO 17)
- **RX (White):** Connect to ESP32-S3 TX (e.g., GPIO 18)
- **TOUCH / WAK (Blue):** Connect to ESP32-S3 interrupt pin (e.g., GPIO 4) for waking from deep sleep
- **3.3V_TOUCH (Green):** Connect to 3.3V

3. Example Code Structure (src/main.cpp)

```
#include <Arduino.h>
#include <Adafruit_Fingerprint.h>

// Use HardwareSerial 1 or 2 on ESP32-S3
HardwareSerial mySerial(1);
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);

#define FINGER_RX_PIN 17
#define FINGER_TX_PIN 18

void setRingLED(uint8_t color, uint8_t mode) {
  // Mode: 1=breathing, 2=flashing, 3=always on, 4=always off
  // Color: 1=red, 2=blue, 3=purple, 4=green, 5=yellow, 6=cyan, 7=white
  finger.LEDcontrol(mode, 100, color, 1);
}

void handleFingerAction(uint16_t fingerID) {
  switch (fingerID) {
    case 1: // Right Index
      Serial.println("Action 1: Unlocking Primary Door...");
      setRingLED(4, 1); // Green breathing LED
      break;

    case 2: // Right Thumb
      Serial.println("Action 2: Toggling Home Assistant Scene...");
      setRingLED(2, 1); // Blue breathing LED
      break;

    case 3: // Left Index (Silent Alarm / Panic Finger)
      Serial.println("Action 3: Triggering Silent Alert...");
      setRingLED(1, 2); // Red flashing LED
      break;
  }
}
```

```

        default:
            Serial.printf("Recognized finger ID #%d (Unassigned Action)\n", fingerID);
            break;
    }
}

int getFingerprintID() {
    uint8_t p = finger.getImage();
    if (p != FINGER_OK) return -1;

    p = finger.image2Tz();
    if (p != FINGER_OK) return -1;

    p = finger.fingerFastSearch();
    if (p == FINGER_OK) {
        Serial.printf("Found ID #%d with confidence %d\n", finger.fingerID, finger.confidence);
        return finger.fingerID;
    } else {
        Serial.println("Finger not recognized");
        setRingLED(1, 2); // Red flash for access denied
        return -1;
    }
}

void setup() {
    Serial.begin(115200);
    mySerial.begin(57600, SERIAL_8N1, FINGER_RX_PIN, FINGER_TX_PIN);

    if (finger.verifyPassword()) {
        Serial.println("Found R503 fingerprint sensor!");
        setRingLED(2, 3); // Solid Blue ready state
    } else {
        Serial.println("Did not find fingerprint sensor :(");
        while (1) { delay(1); }
    }
}

```

```
}

void loop() {
  int matchedID = getFingerprintID();
  if (matchedID > 0) {
    handleFingerAction(matchedID);
  }
  delay(100);
}
```

User prompt: Can you provide a C++ enrollment sketch for the R503 on PlatformIO to enroll new finger IDs using the Serial Monitor?

Response: This complete enrollment sketch uses the **Serial Monitor** to prompt for an ID number, guides you through placing and removing your finger twice, and provides visual feedback using the R503's RGB LED ring.

Requirements & Setup

- **Serial Monitor Settings:** Set the baud rate to **115200** and set line ending to **Newline** or **Both NL & CR**.
- **Default Sensor Baud Rate:** Most Grow R503 modules default to **57600** over UART.

Complete Enrollment Sketch (src/main.cpp)

```
#include <Arduino.h>
#include <Adafruit_Fingerprint.h>

// ESP32-S3 Hardware Serial & Pins
HardwareSerial mySerial(1);
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);

#define FINGER_RX_PIN 17 // Connect to R503 TX (Yellow)
#define FINGER_TX_PIN 18 // Connect to R503 RX (White)

// Helper function to control R503 RGB LED Ring
```

```

// Mode: 1=Breathing, 2=Flashing, 3=Always On, 4=Always Off
// Color: 1=Red, 2=Blue, 3=Purple, 4=Green, 5=Yellow, 6=Cyan, 7=White
void setRingLED(uint8_t color, uint8_t mode) {
    finger.LEDcontrol(mode, 100, color, 1);
}

// Read integer from Serial Monitor
uint16_t readIDFromSerial() {
    uint16_t id = 0;
    while (id == 0) {
        while (!Serial.available()) {
            yield();
        }
        id = Serial.parseInt();
    }
    return id;
}

uint8_t enrollFingerprint(uint16_t id) {
    int p = -1;

    // --- STEP 1: FIRST SCAN ---
    Serial.printf("\n[ID #%d] Place finger on sensor...\n", id);
    setRingLED(3, 1); // Purple breathing LED

    while (p != FINGER_OK) {
        p = finger.getImage();
        switch (p) {
            case FINGER_OK:
                Serial.println("-> Image taken successfully.");
                break;
            case FINGER_NOFINGER:
                break; // Keep waiting
            case FINGER_PACKETRECEIVEERR:
                Serial.println("-> Communication error.");

```

```

        break;
    case FINGER_IMAGEFAIL:
        Serial.println("-> Imaging error.");
        break;
    default:
        Serial.println("-> Unknown error.");
        break;
}
yield();
}

p = finger.image2Tz(1);
if (p != FINGER_OK) {
    Serial.println("-> Failed to convert image template 1.");
    setRingLED(1, 2); // Red flash
    return p;
}

// --- STEP 2: REMOVE FINGER ---
Serial.println("-> Remove finger from sensor.");
setRingLED(5, 2); // Yellow flashing LED
delay(1500);
p = 0;
while (p != FINGER_NOFINGER) {
    p = finger.getImage();
    yield();
}

// --- STEP 3: SECOND SCAN ---
Serial.printf("[ID #%d] Place SAME finger again...\n", id);
setRingLED(3, 1); // Purple breathing LED
p = -1;
while (p != FINGER_OK) {
    p = finger.getImage();
    if (p == FINGER_OK) {

```

```

    Serial.println("-> Second image taken successfully.");
}
yield();
}

p = finger.image2Tz(2);
if (p != FINGER_OK) {
    Serial.println("-> Failed to convert image template 2.");
    setRingLED(1, 2); // Red flash
    return p;
}

// --- STEP 4: CREATE & STORE MODEL ---
Serial.printf("-> Creating feature model for ID %d...\n", id);
p = finger.createModel();
if (p == FINGER_OK) {
    Serial.println("-> Fingerprints matched!");
} else if (p == FINGER_ENROLLMISMATCH) {
    Serial.println("-> Error: Fingerprints did not match. Enrollment aborted.");
    setRingLED(1, 2); // Red flash
    return p;
} else {
    Serial.println("-> Error creating model.");
    setRingLED(1, 2); // Red flash
    return p;
}

p = finger.storeModel(id);
if (p == FINGER_OK) {
    Serial.printf(" SUCCESS! ID %d successfully enrolled and saved in flash.\n", id);
    setRingLED(4, 3); // Solid Green success
    delay(2000);
    setRingLED(2, 3); // Solid Blue standby
    return FINGER_OK;
} else {

```



```

    Serial.println("-> Error saving model to flash memory.");
    setRingLED(1, 2); // Red flash
    return p;
}
}

void setup() {
    Serial.begin(115200);
    while (!Serial) { delay(10); }

    Serial.println("\n--- R503 Fingerprint Enrollment Utility ---");

    // Initialize UART connection to R503
    mySerial.begin(57600, SERIAL_8N1, FINGER_RX_PIN, FINGER_TX_PIN);

    if (finger.verifyPassword()) {
        Serial.println("R503 Sensor detected!");
        setRingLED(2, 3); // Solid Blue standby
    } else {
        Serial.println("R503 Sensor NOT found! Check wiring and power supply.");
        while (1) { delay(1); }
    }

    // Display sensor memory capacity
    finger.getParameters();
    Serial.printf("Sensor Capacity: %d templates | Security Level: %d\n", finger.capacity, finger.security_level);
}

void loop() {
    Serial.println("\n-----");
    Serial.println("Type an ID number (1 - 200) in Serial Monitor to enroll:");

    uint16_t id = readIDFromSerial();
    if (id < 1 || id > 200) {
        Serial.println(" Invalid ID! Please choose a number between 1 and 200.");
    }
}

```

```
    return;
}

Serial.printf("Enrolling ID #%d...\n", id);
enrollFingerprint(id);
}
```

Step-by-Step Workflow

1. Upload the code to your **ESP32-S3** via PlatformIO and open the **Serial Monitor** (Ctrl+Alt+S or click the plug icon).
 2. Enter an integer (e.g., 1) and press **Enter**.
 3. **Purple Breathing Ring**: Place your finger on the sensor glass.
 4. **Yellow Flashing Ring**: Remove your finger when prompted.
 5. **Purple Breathing Ring**: Place the *same* finger on the sensor again.
 6. **Green Ring**: Success! The template is stored directly in the sensor's internal flash memory under that ID number and persists across power cycles.
-

User prompt: Can you provide a helper sketch to list stored fingerprint IDs or delete specific templates on the R503?

Response: This management sketch provides an interactive Serial Monitor menu to **list occupied ID slots, delete individual fingerprint IDs, or wipe the entire sensor memory database.**

Complete Management Utility Sketch (src/main.cpp)

```
#include <Arduino.h>
#include <Adafruit_Fingerprint.h>

// ESP32-S3 Hardware Serial & Pins
HardwareSerial mySerial(1);
```

```

Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);

#define FINGER_RX_PIN 17 // Connect to R503 TX (Yellow)
#define FINGER_TX_PIN 18 // Connect to R503 RX (White)

// Helper function to control R503 RGB LED Ring
// Mode: 1=Breathing, 2=Flashing, 3=Always On, 4=Always Off
// Color: 1=Red, 2=Blue, 3=Purple, 4=Green, 5=Yellow, 6=Cyan, 7=White
void setRingLED(uint8_t color, uint8_t mode) {
    finger.LEDcontrol(mode, 100, color, 1);
}

// Clear leftover characters from Serial buffer
void clearSerialBuffer() {
    while (Serial.available() > 0) {
        Serial.read();
    }
}

// Read an integer input from Serial Monitor
uint16_t readIntegerFromSerial() {
    while (!Serial.available()) {
        yield();
    }
    uint16_t val = Serial.parseInt();
    clearSerialBuffer();
    return val;
}

// -----
// 1. LIST ALL STORED FINGERPRINT IDS
// -----
void listStoredIDs() {
    Serial.println("\n--- Scanning Stored Fingerprint Templates ---");
    setRingLED(5, 2); // Yellow flashing while scanning

```

```

finger.getTemplateCount();
Serial.printf("Total enrolled templates reported: %d / %d\n\n", finger.templateCount, finger.capacity);

if (finger.templateCount == 0) {
    Serial.println("No fingerprints found in memory.");
    setRingLED(2, 3); // Solid Blue
    return;
}

uint16_t foundCount = 0;
Serial.print("Occupied IDs: [ ");

// Query each ID slot individually in sensor flash memory
for (uint16_t id = 1; id <= finger.capacity; id++) {
    if (finger.loadModel(id) == FINGER_OK) {
        Serial.printf("#%d ", id);
        foundCount++;
    }
    yield();
}

Serial.println("]");
Serial.printf("Done! Displayed %d active slots.\n", foundCount);
setRingLED(4, 2); // Green flash
delay(1500);
setRingLED(2, 3); // Solid Blue
}

// -----
// 2. DELETE A SPECIFIC FINGERPRINT ID
// -----
void deleteSpecificID() {
    Serial.println("\nEnter the ID number (1 - 200) you want to DELETE:");
    uint16_t id = readIntegerFromSerial();

```

```

if (id < 1 || id > finger.capacity) {
    Serial.println(" Invalid ID range.");
    setRingLED(1, 2); // Red flash
    delay(1500);
    setRingLED(2, 3);
    return;
}

// Verify if ID actually exists before deleting
if (finger.loadModel(id) != FINGER_OK) {
    Serial.printf(" ID #%d is empty or not found.\n", id);
    setRingLED(1, 2); // Red flash
    delay(1500);
    setRingLED(2, 3);
    return;
}

Serial.printf("Deleting Fingerprint ID #%d...\n", id);
uint8_t p = finger.deleteModel(id);

if (p == FINGER_OK) {
    Serial.printf(" SUCCESS: ID #%d deleted from memory.\n", id);
    setRingLED(4, 2); // Green flash
} else {
    Serial.printf(" Error deleting ID #%d (Code 0x%02X)\n", id, p);
    setRingLED(1, 2); // Red flash
}

delay(1500);
setRingLED(2, 3); // Solid Blue
}

// -----
// 3. WIPE ALL STORED FINGERPRINTS

```

```

// -----
void wipeEntireDatabase() {
  Serial.println("\n WARNING: ARE YOU SURE YOU WANT TO DELETE ALL FINGERPRINTS?");
  Serial.println("Type '99' to CONFIRM wipe, or any other number to cancel:");

  uint16_t confirm = readIntegerFromSerial();

  if (confirm != 99) {
    Serial.println("Wipe operation cancelled.");
    setRingLED(2, 3);
    return;
  }

  Serial.println("Erasing entire sensor database...");
  setRingLED(1, 2); // Red flashing warning

  uint8_t p = finger.emptyDatabase();

  if (p == FINGER_OK) {
    Serial.println(" SUCCESS: Database completely wiped! All fingerprint templates removed.");
    setRingLED(4, 3); // Solid Green
  } else {
    Serial.printf(" Error emptying database (Code 0x%02X)\n", p);
    setRingLED(1, 2); // Red flash
  }

  delay(2000);
  setRingLED(2, 3); // Solid Blue
}

// -----
// SETUP & MAIN MENU LOOP
// -----
void setup() {
  Serial.begin(115200);

```

```

while (!Serial) { delay(10); }

Serial.println("\n--- R503 Fingerprint Database Manager ---");

// Initialize UART connection to R503
mySerial.begin(57600, SERIAL_8N1, FINGER_RX_PIN, FINGER_TX_PIN);

if (finger.verifyPassword()) {
    Serial.println("R503 Sensor connected!");
    setRingLED(2, 3); // Solid Blue standby
} else {
    Serial.println("R503 Sensor NOT found! Check wiring and power supply.");
    while (1) { delay(1); }
}

finger.getParameters();
}

void loop() {
    Serial.println("\n=====");
    Serial.println("SELECT AN OPTION:");
    Serial.println(" [1] List all stored fingerprint IDs");
    Serial.println(" [2] Delete a specific fingerprint ID");
    Serial.println(" [3] WIPE entire fingerprint database");
    Serial.println("=====");
    Serial.print("Enter choice (1-3): ");

    uint16_t choice = readIntegerFromSerial();

    switch (choice) {
        case 1:
            listStoredIDs();
            break;
        case 2:
            deleteSpecificID();
    }
}

```

```
        break;
    case 3:
        wipeEntireDatabase();
        break;
    default:
        Serial.println("Invalid choice. Enter 1, 2, or 3.");
        break;
}
}
```

Serial Monitor Operations

1. Set the Serial Monitor baud rate to **115200** and line ending to **Newline**.
 2. **Option 1:** Iterates through all internal flash memory slots and prints exact ID indices (e.g., #1 #2 #5) currently occupied by templates.
 3. **Option 2:** Asks for a specific ID number, checks if it exists, and removes that single template without touching others.
 4. **Option 3:** Prompts for safety verification code (99) before executing `emptyDatabase()` to return the module to factory-blank state.
-